

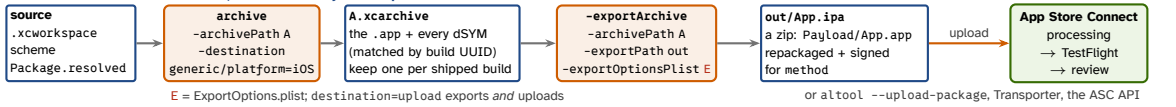
In one line: xcodebuild runs one **action** of a **scheme** against a **destination**; archive + `-exportArchive` with an **ExportOptions.plist** make the shippable `.ipa`; test results land in an `.xcresult` read with `xcresulttool`. `xcrun simctl` scripts simulators, `devicectl` real devices, and `xcode-select / DEVELOPER_DIR` decide **which Xcode** answers. Xcode Cloud runs the same xcodebuild.

One invocation — a workspace needs `-scheme`; everything else has a default

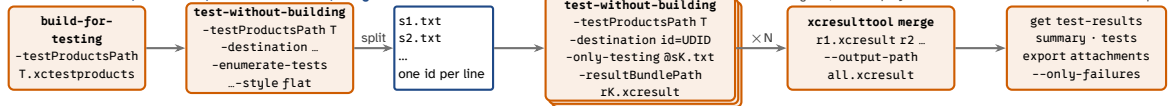
xcodebuild `-workspace A.xcworkspace` `-scheme App` `-configuration Debug` `-destination 'platform=iOS Simulator,name=iPhone 17,OS=26.4'` `-resultBundlePath r.xcresult`

container (or -project) targets, test plans setting set keys platform, id xor name, OS (simulator; default latest), arch, variant; generic/platform=iOS needs no device build setting: beats the target's; -xcconfig f beats it, XCODE_XCONFIG_FILE beats all results; path must not exist `-test` K=V action

Release — each arrow a command, each box a file you can open



Tests — build once, enumerate, shard across Macs, merge the results



Actions

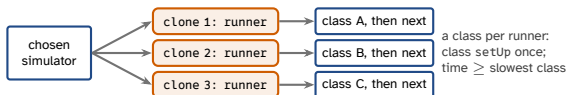
build (default) · build-for-testing (+ `-xcresultrun`) · test · test-without-building · archive · analyze · docbuild · clean; they chain (clean build), build-for-testing, test, archive, analyze need `-scheme`. **Discover:** `-list -json, -showdestinations, -showTestPlans, -showBuildSettings, -showBuildTimingSummary` times each command; `-quiet` = warnings and errors only.

Choosing, repeating and timing tests

-testPlan P -only-test-configuration C	plan name without extension
-only-testing T/C/m -skip-testing ...	@file: one id a line; only wins
-enumerate-tests -format json	list, don't run; needs -destination
-test-iterations N	N runs — or the cap for:
-retry-tests-on-failure	until a pass, ≤3: a flake goes green
-run-tests-until-failure	until a fail, ≤100: finds flakes
-test-repetition-relaunch-enabled YES	fresh process per repetition
-test-timeouts-enabled YES	+ per-test-time-allowance flags
-collect-test-diagnostics on-failure	sysdiagnoses, log archives
-enableCodeCoverage YES -testLanguage	override scheme / test plan

TEST_RUNNER_Foo=Bar → Foo=Bar in the runner. Xcode 26+: `-xcresultrun` keeps disabled plan configurations (`-only-test-configuration`).

Parallel: clones within a destination, suites across destinations



-parallel-testing-enabled YES; -parallel-testing-worker-count N = exactly N. Several -destinations each run the *whole* suite, concurrently; -parallelize-tests-among-destinations splits classes instead. A device lookup waits -destination-timeout (30s).

Result bundles — xcrun xcresulttool

get test-results summary · tests	the summary · the test tree
get test-results test-details · activities	one test: --test-id ID
get test-results insights · metrics	patterns · performance
get build-results · get log --type build	warnings, errors · raw logs
export attachments --only-failures	screenshots, recordings
merge a b --output-path m · compare	combine shards · diff vs --baseline-path
xcrun xcov view --report --json R	coverage per target / file

JSON out; help `get test-results summary` prints the schema. **Xcode 16:** get object fails unless `--legacy` (deprecated) — old parsers broke.

What changed, Xcode 15 → 27

- 15 • -enumerate-tests; -exportArchive uploads with an API key; devicectl
- 16 • get test-results; an unmatched -destination no longer silently runs on the first compatible one; -downloadPlatform -exportPath; devicectl --console
- 26 • Swift 6.2; explicit modules on (SWIFT_ENABLE_EXPLICIT_MODULES=NO); opt-in local compilation cache (COMPILATION_CACHE_ENABLE_CACHING=YES)
- 26.4 • Swift 6.3; runtime export adds · exportBundle; simctl runtime dyld_shared_cache update
- 27 • Swift 6.4; Apple-silicon Macs, macOS 26.6+; -ld_classic gone; storyboards compile with no simulator; simctl/devicectl reboot; devicectl --json-output to stdout

Traps

- ! xcodebuild ... | xcbeautify without set -o pipefail: the pipeline returns the formatter's status — a red build reports green.
- ! No OS= means latest: a new runner image silently moves the iOS version — pin OS= or id=.
- ! Exit codes (sysxits): **70 = build or test failed**; 64 usage, 66 no input, 74 I/O.

ExportOptions.plist — what -exportArchive reads

method	app-store-connect · release-testing (ad hoc) · enterprise · debugging (default) · developer-id · mac-application · validation; app-store, ad-hoc, development are deprecated
destination	export (local file) or upload (to App Store Connect)
manageAppVersion	Xcode rewrites the build number of everything in the archive — turn off when CI owns the number
AndBuildNumber	team-only TestFlight; the build can never be submitted
testFlightInternal	
TestingOnly	
signingStyle teamID	manual needs provisioningProfiles (bundle id → profile); uploadSymbols for symbolicated crashes

Signing on a runner: -allowProvisioningUpdates fetches profiles (and creates certificates for automatic signing) through an account in Xcode's Accounts pane — or an App Store Connect API key: -authenticationKeyPath, -authenticationKeyID, -authenticationKeyIssuerID (also for uploads, Xcode 15+). alttool looks for AuthKey_<id>.p8 in `./private_keys, ~/ .appstoreconnect/private_keys` and two more.

Swift packages, plugins, macros on CI

- Commit Package.resolved (in App.xcodeproj/project.xcworkspace/xcsharedata/swiftpm/); -disableAutomaticPackageResolution makes CI use exactly it.
- Checkouts land in DerivedData/SourcePackages unless -clonedSourcePackagesDirPath P: pin it to cache them.
- Private repos: SSH URLs + known_hosts; -scmProvider system uses the Mac's git config (URL rewrites, proxies, SSH); registries: -packageAuthorizationProvider netrc.
- Headless, nobody clicks "Trust & Enable": -skipPackagePluginValidation, -skipMacroValidation — "can be a security risk".

xcrun simctl — the iOS userspace on the Mac kernel, scripted

List --json devices · create CI <type> <runtime>	create prints the UDID
boot U · bootstatus U -b (unverified) · shutdown all	wait until usable
erase U · clone U base · delete unavailable	erase, clone: shut down
install booted A.app · launch booted id -K v	-K v: a default, once
launch --console-pty · spawn booted log stream	stdio, logs in terminal
openurl booted https://... · push booted p.apns	aps JSON; id may be in it
privacy booted grant photos id	also revoke, reset all
keychain booted add-root-cert ca.pem	trust your CA / proxy
status_bar booted override --time 9:41	clear undoes it
ui booted appearance dark · addmedia booted p.jpg	dark mode · photos
io booted screenshot · recordVideo --codec h264	CI evidence
get_app_container booted id · runtime add R.dmg	sandbox path · import

booted = the one booted device; script with UDIDs, not names. **Devices:** xcrun devicectl (CoreDevice, iOS 17+): list devices, device install app --device D A.app, device process launch --console id; --device takes a name, UDID or ECID (Xcode 16).

Which Xcode, which components

sudo xcode-select -s /Applications/Xcode_27.app: every user, persistent; DEVELOPER_DIR=~/Xcode_27.app: this process only. /usr/bin/xcodebuild, git, swift are shims through it; xcrun --find T resolves a tool. xcodes install 27.0; pin the version in .xcode-version. Fresh Mac: -runFirstLaunch (components + licence) · -downloadPlatform iOS -exportPath D [-buildVersion 26.4] once, -importPlatform D/... .dmg on each Mac · -downloadComponent metalToolchain.

Xcode Cloud

From ci_scripts/ beside the project it runs only ci_post_clone.sh, ci_pre_xcodebuild.sh, ci_post_xcodebuild.sh (runs even if xcodebuild failed). No shebang → zsh file; no sudo; created files are deleted; non-zero exit fails the build. Scripts read CI_* variables (CI_PULL_REQUEST_NUMBER ...); secret ones print as ****. 25 compute h/month come with the Developer Program; plans add 100, 250, 1,000 h.

Related: ci-pipeline-design · fastlane-deep · github-actions-deep · Code signing & iOS CI/CD · macos-keychain-security-cli · Flaky tests · CI · coverage · TDD · xctest-and-swift-testing · Crashes & symbolication · Modularization & Swift Package Manager · macos-power-user-cli